

LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 3 – Konsep Dasar Pemrograman Berorientasi Objek



NAMA : Ahnaf Ariacheda

NIM : 10313240037

KELAS : PS1DS-05-REG-01

PROGRAM STUDI SAINS DATA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Pemrograman Berorientasi Objek (OOP) adalah sistem yang dibangun dengan mengenkapsulasi komponen menjadi kelompok data dan fungsi yang saling berinteraksi. OOP memiliki empat karakteristik utama: Abstraksi (menemukan atribut dan fungsi esensial), Enkapsulasi (menyembunyikan rincian data untuk proteksi), Pewarisan (mewariskan sifat ke kelas turunan), dan Polimorfisme (satu antarmuka dengan banyak aksi).

Dalam OOP, Kelas berfungsi sebagai cetak biru (*blueprint*), sedangkan Objek adalah hasil ciptaan (instansiasi) dari kelas tersebut. Kelas memiliki atribut yang disebut *Field*:

- **Class Field:** Menggunakan *modifier* `static`, nilainya terikat pada kelas dan dibagi ke seluruh objek.
- **Instance Field:** Dideklarasikan tanpa `static`, di mana tiap objek memiliki salinan nilainya sendiri.

Objek berinteraksi melalui *Method* (fungsi/prosedur) dengan dua cara pelewatan parameter:

- **Pass by Value:** Berlaku untuk tipe data primitif, di mana program hanya menyalin isi memori sehingga nilai asli tidak berubah.
- **Pass by Reference:** Berlaku untuk *array* dan objek, di mana yang dilewatkan adalah alamat memori, sehingga perubahan di dalam *method* akan langsung memengaruhi nilai aslinya.

Terakhir, Java menggunakan *keyword* `this` sebagai referensi khusus untuk merujuk pada objek yang sedang aktif berjalan.

B. Guided

1. Package Abstraksi :

a. Class Abstraksi.java

```
package guided.Abstraksi;

public class Abstraksi {
    public static void main(String[] args) {
        Kucing aren = new Kucing();

        aren.nama = "Mujaer";
        aren.ras = "Aren Gentong";
    }
}
```

```
        aren.bersuara();  
    }  
}
```

b. Class Kucing.java

```
package guided.Abstraksi;  
  
public class Kucing {  
    String nama;  
    String ras;  
  
    void bersuara() {  
        System.out.println(nama + " " + ras + " meow");  
    }  
}
```

Output :

```
PS C:\Users\ahnaf ariacheda\Music\Praktikum PBO Java> & 'C:\Users\ahnaf ariacheda\.vscode\extensions\redhat.java-1.56.0-win32-x64\jre\21.0.12.1-win32-x86_64\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\ahnaf ariacheda\Music\Praktikum PBO Java\praktikum-pbo-ps1ds-05-reg-01\modul-03-konsep-dasar\project\modul03\target\classes' 'guided.Abstraksi.Abstraksi'  
Mujaer Aren Gentong meow
```

2. Package enkapsulasi :

a. Class Enkapsulasi.java

```
package guided.enkapsulasi;  
  
public class Enkapsulasi {  
    public static void main(String[] args) {  
        Rekening rek = new Rekening();  
    }  
}
```

```

        rek.tambahSaldo(20000000);

        rek.tampilkanSaldo();

    }

}

```

b. Class Rekening.java

```

package guided.enkapsulasi;

public class Rekening {
    private int saldo = 0;

    public void tambahSaldo(int jumlah) {
        saldo = saldo + jumlah;
        System.out.println("Saldo berhasil ditambahkan.");
    }

    public void tampilkanSaldo() {
        System.out.println("Saldo saat ini: " + saldo);
    }
}

```

Output :

```

PS C:\Users\ahnaf ariacheda\Music\Praktikum PBO Java> & 'C:\Users\ahnaf ariacheda\.vscode\extensions\redhat.java-1.56.0-win32-x64\jre\21.0.12.1-win32-x86_64\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\ahnaf ariacheda\Music\Praktikum PBO Java\praktikum-pbo-pslds-05-reg-01\modul-03-konsep-dasar\project\modul03\target\classes' 'guided.enkapsulasi.Enkapsulasi'
Saldo berhasil ditambahkan.
Saldo saat ini: 20000000

```

3. circle01.java

```

package guided;

public class circle01 {
    public static final double PI = 3.14159;

    public static double radiansToDegrees(double radians) {

```

```

        return radians * (180.0 / PI);
    }

    public double r;

    public double area(){
        return PI * r * r;
    }

    public double circumference(){
        return 2 * PI * r;
    }
}

```

C. Unguided

1. Soal Unguided 1

Sebuah tim mengumpulkan suhu harian suatu kota selama satu minggu (7 hari) dari sensor otomatis. Data ini sudah tersedia sebagai array di `main` sebelum diserahkan untuk diolah.

Pada salah satu hari, sensor gagal mengirim data. Hari yang gagal terekam ditandai dengan sebuah nilai penanda pada array, karena suhu asli tidak mungkin sebesar itu. Nilai penanda ini sama untuk semua data suhu yang diolah kapan pun dan oleh siapa pun – bukan sesuatu yang berbeda-beda tiap kali data dibuat – sehingga sebaiknya ditulis satu kali saja sebagai aturan tetap milik class, bukan diketik ulang sebagai angka `-1.0` di setiap method yang membutuhkannya.

Buatlah satu class yang bertugas mengolah data suhu ini – memeriksa apakah ada data kosong, mengisinya, dan menghasilkan statistik dasar dari data yang sudah bersih.

Jawaban :

- package unguided.suhu
 - a. PengolahSuhu.java

```
package unguided.suhu;

public class PengolahSuhu {

    // Class field (static final): aturan tetap milik class dan dipakai bersama
    public static final double NILAI_KOSONG = -1.0;

    // Instance field: menyimpan array suhu 7 hari yang di-enkapsulasi (private)
    private double[] suhuHarian;

    // Constructor menggunakan nama parameter yang sama dengan field sehingga memakai 'this'
    public PengolahSuhu(double[] suhuHarian) {
        this.suhuHarian = suhuHarian;
    }

    // Method untuk menampilkan seluruh data suhu apa adanya
    public void tampilkanData() {
        for (int i = 0; i < suhuHarian.length; i++) {
            if (suhuHarian[i] == NILAI_KOSONG) {
                System.out.println("Hari " + (i + 1) + " : (kosong)");
            } else {
                System.out.println("Hari " + (i + 1) + " : " + suhuHarian[i] + "°C");
            }
        }
    }

    // Method untuk mencari index data yang bernilai NILAI_KOSONG
```

```
public int cariIndexKosong() {  
    for (int i = 0; i < suhuHarian.length; i++) {  
        if (suhuHarian[i] == NILAI_KOSONG) {  
            return i; // Mengembalikan index ditemukannya data kosong  
        }  
    }  
    return -1; // Mengembalikan -1 jika tidak ada data kosong  
}  
  
// Method untuk mengisi data kosong berdasarkan rata-rata tetangganya  
public void isiDataKosong() {  
    int index = cariIndexKosong(); // Memanggil method cariIndexKosong()  
  
    // Memastikan index ditemukan dan berada di antara index awal dan akhir  
    if (index != -1 && index > 0 && index < suhuHarian.length - 1) {  
        // Rumus imputasi: (suhuHarian[i-1] + suhuHarian[i+1]) / 2  
        this.suhuHarian[index] = (this.suhuHarian[index - 1] + this.suhuHarian[index + 1]) / 2.0;  
    }  
}  
  
// Method untuk menghitung rata-rata dari data yang sudah bersih  
public double hitungRataRata() {  
    double total = 0;  
    for (double suhu : suhuHarian) {  
        total += suhu;  
    }  
}
```

```
        return total / suhuHarian.length;
    }
}
```

b. main.java

```
package unguided.suhu;

import java.util.Arrays;

public class main {
    public static void main(String[] args) {
        // 1. Data suhu harian mentah[cite: 1]
        double[] suhuHarian = { 30.4, 24.3, 26.8, -1.0, 31.4, 30.8, 32.9 };

        // 2. Inisialisasi object PengolahSuhu[cite: 1]
        PengolahSuhu pengolah = new PengolahSuhu(suhuHarian);

        System.out.println("=== Data Suhu Awal ===");
        pengolah.tampilkanData(); // Tampilkan data awal[cite: 1]
        System.out.println();

        // Cari dan tampilkan index hari kosong[cite: 1]
        int indexKosong = pengolah.cariIndexKosong();
        System.out.println("Index hari kosong (dimulai dari 0): " + indexKosong);
        System.out.println();

        // 3. Jalankan imputasi pengisian data[cite: 1]
        pengolah.isiDataKosong();

        // 4. Tampilkan data akhir yang sudah bersih beserta statistik rata-rata[cite: 1]
        System.out.println("=== Data Suhu Setelah Pengisian ===");
        pengolah.tampilkanData();
        System.out.println();

        System.out.printf("Rata-rata : %.2f°C\n", pengolah.hitungRataRata());
    }
}
```

```

System.out.println();

// 5. Cetak array dari variabel main langsung[cite: 1]
System.out.println("Isi array suhuHarian di main setelah isiDataKosong() dijalankan:");
System.out.println(Arrays.toString(suhuHarian));
System.out.println("(ikut berubah: constructor menyimpan referensi array yang sama)");
}
}

```

Output :

```

=== Data Suhu Awal ===

```

```

Hari 1 : 30.4°C
Hari 2 : 24.3°C
Hari 3 : 26.8°C
Hari 4 : (kosong)
Hari 5 : 31.4°C
Hari 6 : 30.8°C
Hari 7 : 32.9°C

```

```

Index hari kosong (dimulai dari 0): 3

```

```

=== Data Suhu Setelah Pengisian ===

```

```

Hari 1 : 30.4°C
Hari 2 : 24.3°C
Hari 3 : 26.8°C
Hari 4 : 29.1°C
Hari 5 : 31.4°C
Hari 6 : 30.8°C
Hari 7 : 32.9°C

```

```

Rata-rata : 29.39°C

```

```

Isi array suhuHarian di main setelah isiDataKosong() dijalankan:
[30.4, 24.3, 26.8, 29.1, 31.4, 30.8, 32.9]
(ikut berubah: constructor menyimpan referensi array yang sama)

```

Penjelasan :

- **Konteks Program:** Program ini dirancang untuk mengolah data deret waktu berupa suhu harian selama 7 hari dari sensor otomatis, mendeteksi kegagalan rekam, dan membersihkan data tersebut.
- **Penerapan Field & Enkapsulasi:** Array `suhuHarian` diatur sebagai *instance field* privat agar datanya terkunci dan tidak bisa dimanipulasi langsung dari luar kelas. Sebaliknya, penanda gagal rekam `-1.0` dijadikan *class field* (`public static final NILAI_KOSONG`) sebagai aturan tetap yang dipakai bersama, sehingga tidak perlu diketik berulang di setiap *method*.
- **Penggunaan Keyword `this`:** Dipakai wajib secara eksplisit di dalam *constructor* untuk membedakan antara atribut milik kelas dan parameter *input* yang kebetulan memiliki nama yang persis sama.
- **Logika Imputasi Data:** Saat *method* menemukan hari dengan `NILAI_KOSONG`, program melakukan pemulihan data menggunakan metode imputasi dasar, yaitu menjumlahkan suhu dari tetangga terdekatnya (hari sebelum dan sesudah) lalu dibagi dua.

- **Konsep *Pass-by-Reference*:** Manipulasi data yang dilakukan oleh `isiDataKosong()` pada objek otomatis membuat *array* di `main` ikut berubah. Hal ini terjadi karena tipe data *array* di Java disalurkan menggunakan referensi; *constructor* menyimpan alamat memori yang sama persis dengan *array* di program utama, bukan membuat salinan datanya.

D. Kesimpulan

Berdasarkan praktikum Modul 3 ini, dapat disimpulkan bahwa Pemrograman Berorientasi Objek (OOP) adalah metode yang mengenkapsulasi data beserta fungsinya ke dalam sebuah kelas untuk menciptakan suatu objek. Konsep penting yang diterapkan dalam praktikum ini meliputi pemahaman perbedaan *Class Field* (atribut statis milik kelas) dan *Instance Field* (atribut unik milik tiap objek), serta prinsip enkapsulasi (*data hiding*). Selain itu, praktikum ini membuktikan adanya konsep *Pass by Reference*, di mana objek *array* yang dimasukkan ke dalam *constructor* tidak disalin nilainya, melainkan dikirim alamat memorinya. Hal ini memungkinkan perubahan data di dalam sebuah objek (*method*) secara otomatis memengaruhi variabel aslinya di program utama.